

Nuée d'Oiseaux

Simulation multi-agents en OCaml fonctionnel pur

Ce document présente l'architecture, les algorithmes et les choix d'implémentation d'une simulation de type **boids** développée en OCaml pur, sans effets de bord ni état mutable. Le modèle boids, introduit par Craig Reynolds en 1986, reproduit le comportement collectif d'une nuée d'oiseaux à partir de trois règles locales simples appliquées à chaque agent.

1. Architecture du projet

Le projet est découpé en six modules OCaml indépendants. Les flèches indiquent les dépendances entre modules (pointillés = import / open).

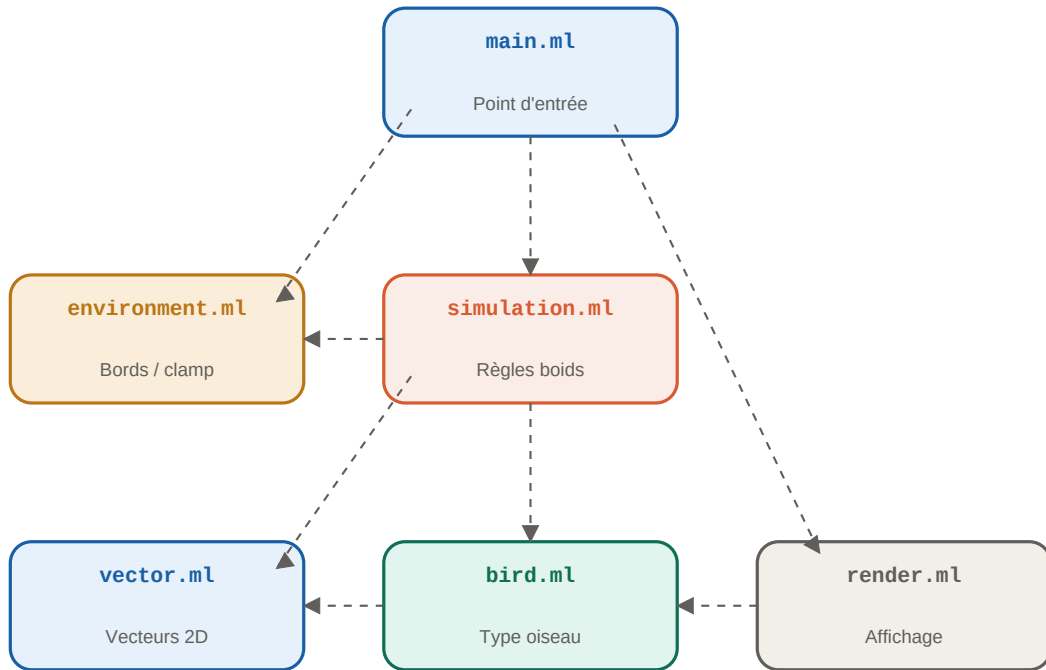


Figure 1 — Dépendances entre les modules OCaml

- **vector.ml** — Bibliothèque de vecteurs 2D : addition, soustraction, mise à l'échelle, norme, normalisation, limitation. Aucune dépendance externe.
- **bird.ml** — Type `bird` composé d'une position et d'une vitesse (deux vecteurs). Fournit la génération aléatoire et le déplacement d'un oiseau.
- **environment.ml** — Dimensions de la fenêtre, force de répulsion des bords et clamp de sécurité pour garder les oiseaux dans la fenêtre.
- **simulation.ml** — Cœur algorithmique : calcul des trois règles bords et mise à jour de la liste d'oiseaux à chaque pas de temps.
- **render.ml** — Affichage avec la bibliothèque Graphics : efface l'écran et dessine chaque oiseau (disque + segment de vitesse).
- **main.ml** — Initialisation et boucle infinie de simulation. Appuyer sur Échap pour quitter.

2. Les trois règles bords

Chaque oiseau observe ses voisins dans un rayon de perception et calcule trois forces d'accélération qu'il somme pour obtenir sa nouvelle vitesse. Ces règles sont purement locales : aucun oiseau n'a une vue globale du groupe.

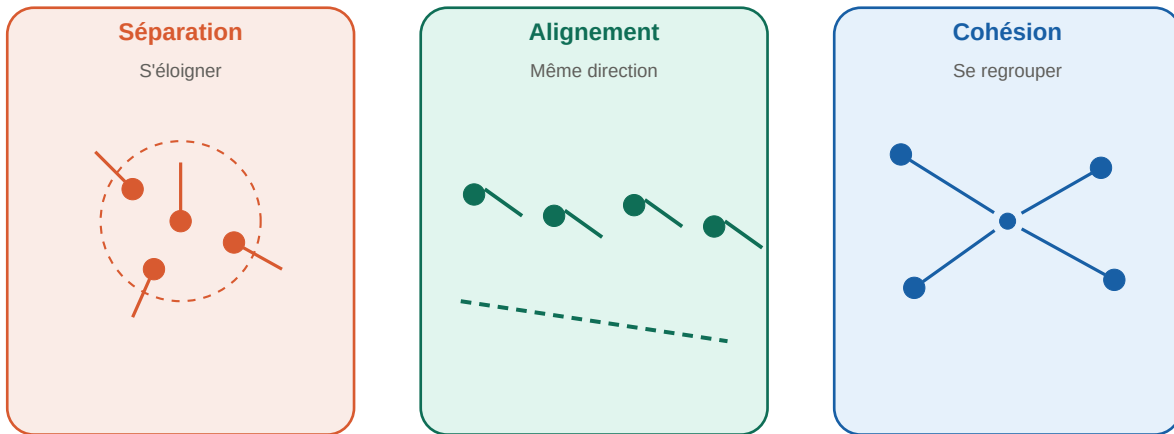


Figure 2 — Séparation, alignement et cohésion

2.1 Séparation

Chaque oiseau calcule un vecteur de fuite pondéré par l'inverse de la distance pour chaque voisin situé dans le *separation_radius*. Plus un voisin est proche, plus la force de répulsion est forte. Le résultat est normalisé puis limité par *max_force*.

2.2 Alignement

L'oiseau calcule la moyenne des vitesses de tous ses voisins dans le *perception_radius*, l'utilise comme direction désirée et produit une force de correction (différence entre désir et vitesse actuelle), limitée par *max_force*.

2.3 Cohésion

L'oiseau se dirige vers le centre de masse de ses voisins. La force désirée pointe vers ce centre à vitesse maximale ; la force appliquée est la correction nécessaire pour y parvenir.

3. Boucle de simulation

La simulation tourne en boucle récursive terminale. À chaque pas, la liste d'oiseaux est transformée de façon purement fonctionnelle (`List.map`) : aucun état mutable n'est modifié.

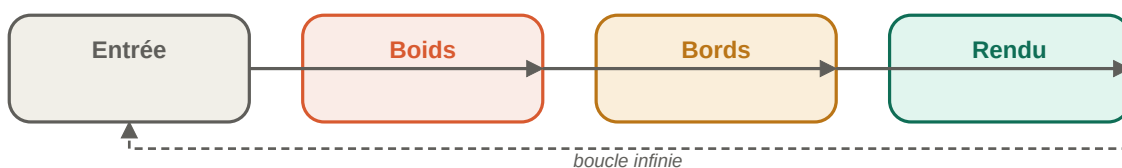


Figure 3 — Pipeline d'un pas de simulation

Le pipeline d'un pas de temps se déroule ainsi :

- Calcul des trois forces boids pour chaque oiseau (en observant la liste courante)
- Application de la force de répulsion des bords
- Limitation de la vitesse à `max_speed`
- Déplacement : nouvelle position = ancienne position + nouvelle vitesse
- Clamp de sécurité pour maintenir les oiseaux dans la fenêtre
- Rendu graphique, puis attente de 16 ms (≈ 60 fps)

4. Gestion des bords

Plutôt que de téléporter les oiseaux d'un bord à l'autre (wrap), le module `environment.ml` applique une **force de rappel** progressive dès qu'un oiseau entre dans la zone marginale (50 px des bords). Cela produit des virages naturels sans transition brutale.

La fonction `c_lamp` est un filet de sécurité : elle restreint les coordonnées à $[1, \text{largeur}-1] \times [1, \text{hauteur}-1]$ pour le cas où un oiseau traverserait malgré tout le bord.

5. Paramètres de réglage

Tous les paramètres sont regroupés en tête de `simulation.ml` et `environment.ml` pour faciliter l'expérimentation :

Paramètre	Valeur	Rôle
<code>perception_radius</code>	60.	Rayon de vision de chaque oiseau
<code>separation_radius</code>	20.	Distance minimale entre oiseaux
<code>max_speed</code>	3.	Vitesse maximale
<code>max_force</code>	0.05	Force de virage maximale
<code>sep_weight</code>	1.5	Poids de la séparation
<code>align_weight</code>	1.0	Poids de l'alignement
<code>cohes_weight</code>	1.0	Poids de la cohésion
<code>margin</code>	50.	Zone de répulsion des bords (px)
<code>turn_factor</code>	0.3	Force de rappel vers l'intérieur

Table 1 — Paramètres et valeurs par défaut

6. Compilation et exécution

Le projet utilise **Dune** comme système de build. La bibliothèque `graphics` doit être installée via `opam`.

```
# Installer les dépendances
opam install graphics

# Compiler
dune build

# Lancer
dune exec ./main.exe

# Quitter : appuyer sur Echap
```

Le fichier `dune` déclare l'exécutable avec ses deux bibliothèques :

```
(executable
 (name main)
 (modules main vector bird environment simulation render)
 (libraries graphics unix))
```

7. OCaml fonctionnel pur

L'implémentation respecte strictement le paradigme fonctionnel :

- Pas de `ref`, de tableaux mutables ni de variables globales
- `List.map` et `List.fold_left` remplacent les boucles impératives
- Chaque fonction retourne une nouvelle valeur sans modifier ses arguments
- La boucle principale est une récursion terminale
- Les types `vector`, `bird` et `environment` sont des records immuables

Cette approche garantit l'absence d'effets de bord non contrôlés et rend chaque fonction individuellement testable.